



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **informatyka**

Daniel Wiszniowski

nr albumu: 318635

Projekt i realizacja bezakumulatorowych czujników klimatycznych zasilanych słonecznie

Design and development of battery-free
solar-powered climate sensors

Praca licencjacka

napisana w Katedrze Oprogramowania Systemów Informatycznych

Instytutu Informatyki i Matematyki UMCS

pod kierunkiem **dr hab. Beaty Byliny, prof. UMCS**

Lublin 2026

Spis treści

Wstęp	5
1 Czujniki klimatyczne	7
1.1 Moduły pomiarowe	8
1.2 Komunikacja z modułami	9
1.3 Zestawienie wybranych modułów	9
1.4 Zasilanie czujników klimatycznych	10
2 Programowanie systemów wbudowanych	13
2.1 Charakterystyka układów sterujących	13
2.2 Języki programowania	14
2.2.1 Język C	14
2.2.2 Język C++	14
2.2.3 MicroPython	15
2.2.4 Język Rust	15
2.2.5 Podsumowanie i porównanie cech języków	16
3 Realizacja projektu czujników klimatycznych	17
3.1 Założenia projektowe	17
3.2 Moduły pomiarowe	18
3.3 Zasilanie	18
3.4 Mikrokontroler	18
3.5 Komunikacja radiowa	18
3.6 Oprogramowanie	19
4 Część sprzętowa	21
4.1 Sekcja ładowania superkondensatorów	21
4.2 Układ regulacji napięcia	22
4.3 Mikrokontroler ATmega328P	23
4.4 Magistrala I ² C	24

4.5	Układ komunikacji radiowej	25
4.6	Moduł radiometryczny	25
4.7	Pozostałe elementy pomiarowe	26
4.8	Pozostałe wyprowadzenia	26
5	Oprogramowanie	27
5.1	Narzędzia i biblioteki nieautorskie	27
5.2	Moduł AHT20	28
5.3	Moduł BMP280	33
5.4	Moduł VEML7700	38
5.5	Moduł z licznikiem Geigera-Müllera	43
5.6	Obsługa modułu radiowego	46
5.6.1	Przebieg transmisji	46
5.6.2	Maszyna stanów	46
5.6.3	Struktura Transmitter	47
5.6.4	Przygotowanie danych, kodowanie 4b6b	47
5.6.5	Obsługa maszyny stanów	49
5.7	Serializacja danych	49
5.8	Czujnik klimatyczny	51
5.9	Usypianie mikrokontrolera	52
5.10	Sterowanie zasilaniem	53
5.11	Funkcja główna	54
6	Testy	55
6.1	Zasięg komunikacji radiowej	55
6.2	Pobór prądu	55
6.3	Czas pracy	55
6.4	Porównanie z rozwiązaniami komercyjnymi	55
	Podsumowanie	57
	Spis listingów	59
	Spis tabel	61
	Spis rysunków	63
	Bibliografia	65

Wstęp

Tu treść wstępu (który piszemy na końcu, po napisaniu całej pracy).

Rozdział 1

Czujniki klimatyczne

Rozdział pierwszy wprowadza w tematykę pracy, przedstawiając kontekst zastosowania czujników klimatycznych we współczesnej inżynierii oraz uzasadniając dobór komponentów pomiarowych i energetycznych wykorzystanych w projekcie. Omówiono dostępne na rynku moduły realizujące pomiary temperatury, wilgotności, ciśnienia atmosferycznego i natężenia oświetlenia, a także zagadnienia związane z zasilaniem urządzeń zewnętrznych ze źródeł fotowoltaicznych, w tym porównanie ogniw litowo-jonowych z superkondensatorami jako magazynów energii.

Współczesny rozwój technologiczny w połączeniu z postępującymi zmianami klimatu oraz rosnącą automatyzacją procesów przemysłowych i urządzeń domowych sprawiły, że precyzyjne obserwowanie i dokumentowanie parametrów środowiskowych stało się nieodłącznym elementem współczesnej inżynierii.

Na potrzeby tej pracy czujnikiem klimatycznym określa się urządzenie wyposażone w moduły pomiarowe — takie jak termometr, higrometr czy barometr — zajmujące się akwizycją danych dotyczących parametrów atmosfery w wybranym obszarze. Dane te są kluczowe w wielu dziedzinach życia gospodarczego i społecznego. W rolnictwie precyzyjnym (ang. *smart agriculture*) pozwalają na optymalizację nawadniania i ochrony upraw, w aglomeracjach miejskich służą do monitorowania smogu i poziomu zanieczyszczeń powietrza, natomiast w systemach zarządzania budynkami (ang. *Building Management Systems*) stanowią podstawę sterowania procesami klimatyzacji, wentylacji i ogrzewania.

Na rynku konsumenckim dostępna jest szeroka gama stacji pogodowych będących zbiorem czujników klimatycznych, z których dane przesyłane są do stacji bazowych. Często stacje pogodowe do użytku domowego składają się z samej stacji bazowej, będącej jednocześnie czujnikiem klimatycznym. Do popularnych producentów tego segmentu należą Oregon Scientific [26], TFA Dostmann [33], Netatmo [25] oraz Bresser [16]. Współczesne cyfrowe czujniki klimatyczne wyposażone są w elektroniczne moduły po-

miarowe, oferujące większą precyzję i większą wygodę odczytu danych w porównaniu do ich analogowych odpowiedników. W celu akwizycji danych klimatycznych z różnych miejsc, stacje pogodowe wyposażone są w czujniki komunikujące się radiowo — najczęściej na częstotliwości 433 MHz lub 868 MHz [34] — ze stacją bazową. Stacja bazowa zajmuje się przetworzeniem danych i przekazaniem ich użytkownikowi. Często wyposażona jest we własne moduły pomiarowe oraz dodatkowe funkcje, na przykład zapisywanie danych historycznych. Rozwiązania klasy wyższej, takie jak stacja Netatmo, integrują się z platformami inteligentnego domu i umożliwiają monitorowanie dodatkowych parametrów, jak stężenie CO₂ czy poziom hałasu, prezentując dane za pośrednictwem aplikacji mobilnej. Pomimo rosnącej funkcjonalności, urządzenia konsumenckie charakteryzują się ograniczoną dokładnością pomiarową — precyzyjne modele cyfrowe osiągają granicę błędu temperatury na poziomie 1°C i wilgotności w granicach 3–5%, co w zastosowaniach wymagających wysokiej dokładności lub ciągłej akwizycji danych jest niewystarczające. Stanowi to bezpośrednią motywację do budowy dedykowanego systemu pomiarowego opartego na precyzyjnych modułach elektronicznych, opisanego w niniejszej pracy.

1.1 Moduły pomiarowe

Do podstawowych wielkości fizycznych rejestrowanych przez czujniki klimatyczne zalicza się przede wszystkim temperaturę, wilgotność względną oraz ciśnienie atmosferyczne. Obecne moduły cyfrowe integrują elementy pomiarowe, dedykowane przetworniki analogowo-cyfrowe (ADC, ang. *analog-to-digital converter*) oraz fabryczne obwody kalibracyjne w jednej obudowie. Rozwiązanie takie pozwala wyeliminować problem zakłóceń elektromagnetycznych indukowanych w przewodach sygnałowych. Ponadto projektanci modułów implementują algorytmy linearyzacji charakterystyk sensorów oraz cyfrowe filtry szumów, co umożliwia wygodny odczyt danych bezpośrednio przez mikrokontroler.

Na rynku dostępny jest szereg modułów realizujących pomiary klimatyczne, różniących się zarówno mierzonymi wielkościami fizycznymi, jak i osiąganą dokładnością, zakresem pomiarowym oraz stopniem integracji. Wśród popularnych modułów pomiarowych temperatury i wilgotności wyróżnić można przede wszystkim rodzinę DHT (DHT11 [3], DHT22 [4]) firmy AOSONG, charakteryzującą się niskim kosztem i szeroką dostępnością, lecz ograniczoną dokładnością. Wyższą klasę dokładnościową reprezentują moduły szwajcarskiej firmy Sensirion — seria SHT3 [30] (SHT30, SHT31, SHT35) oraz nowsza seria SHT4x [31] (SHT40, SHT41, SHT45) oferujące dokładność pomiaru temperatury na poziomie $\pm 0,2^\circ\text{C}$. W obszarze pomiaru ciśnienia atmosferycznego dominuje oferta firmy Bosch Sensortec — od popularnego BMP180 [10], przez

powszechnie stosowany BMP280 [15], po nowsze BMP388 [11] i BMP390 [12] charakteryzujące się wyższą rozdzielczością. Firma Bosch oferuje również moduły z rodziny BME (BME280 [13], BME680 [14], BME688 [14]), łączące pomiar ciśnienia, temperatury i wilgotności w jednej obudowie, przy czym moduły BME680 i BME688 wyposażone są dodatkowo w czujnik gazów lotnych (VOC). Do pomiaru natężenia oświetlenia w zakresie widzialnym najczęściej stosuje się moduły BH1750 [27] firmy ROHM oraz VEML7700 [36] firmy Vishay.

1.2 Komunikacja z modułami

Komunikacja z modułami pomiarowymi może odbywać się za pośrednictwem wielu protokołów. Głównymi z nich są:

- **USART** — asynchroniczna lub synchroniczna magistrala 2-kierunkowa, korzystająca z 2 połączeń do transmisji danych — nadawczego (TX) i odbiorczego (RX). W trybie synchronicznym wykorzystywane jest dodatkowe połączenie zegarowe (XCK). Przesyła dane w trybie *full duplex*, w konfiguracji *peer-to-peer*. Zaprojektowana do użytku w krótkich połączeniach, najlepiej wewnątrz płytkowych (ang. *intra-board*), choć przy zastosowaniu standardu RS-232 lub RS-485 możliwa jest transmisja na większe odległości.
- **I²C** — synchroniczna magistrala 2-kierunkowa, korzystająca z 2 połączeń — zegarowego (SCL) i danych (SDA). Wysyłająca dane w trybie *half duplex*, w konfiguracji *master-slave*. Zaprojektowana do użytku w krótkich połączeniach, najlepiej wewnątrz płytkowych (ang. *intra-board*) [17]. Na dłuższych dystansach zachodzi degradacja sygnału (ang. *signal degradation*).
- **1-Wire** — asynchroniczna magistrala 2-kierunkowa, używająca 1 połączenia do 2-kierunkowego przesyłu danych w trybie *half duplex*, w konfiguracji *master-slave*. Obsługuje niskie prędkości przesyłu od 15,4 do 125 kbps. W przeciwieństwie do magistrali I²C, przesył danych jest możliwy na znacznie większe odległości, osiągające nawet 100 m [37].

1.3 Zestawienie wybranych modułów

Zestawienie wybranych, reprezentatywnych modułów dostępnych na rynku wraz z ich podstawowymi parametrami przedstawiono w tabeli 1.1.

Tabela 1.1: Zestawienie wybranych modułów pomiarowych dostępnych na rynku

Moduł	Wielkości mierzone	Interfejs	Uwagi
DHT22 (AM2302)	T, RH	1-Wire	niski koszt
AHT20	T, RH	I ² C	–
SHT31	T, RH	I ² C	wysoka stabilność
SHT40	T, RH	I ² C	najnowsza generacja
DS18B20	T	1-Wire	wodoodporny wariant
BMP180	T, p	I ² C	przestarzały
BMP280	T, p	I ² C/SPI	niski pobór mocy
BME280	T, RH, p	I ² C/SPI	3 wielkości
BME688	T, RH, p, VOC	I ² C/SPI	detekcja gazów
BH1750	E _v	I ² C	do 65 535 lx
VEML7700	E _v	I ² C	do 120 000 lx

T – temperatura, RH – wilgotność względna, p – ciśnienie atmosferyczne, E_v – natężenie oświetlenia.

1.4 Zasilanie czujników klimatycznych

Sposób zasilania czujników klimatycznych jest uzależniony od miejsca ich docelowego zastosowania. W przypadku montażu wewnętrznego możliwe jest wykorzystanie sieciowego zasilacza impulsowego o napięciu wyjściowym dobranym do wymagań układu, jak również jednorazowych baterii alkalicznych. W przypadku montażu zewnętrznego doprowadzenie zasilania przewodem bywa często niepraktyczne lub niemożliwe. Alternatywę stanowią baterie jednorazowe, jednak wymagają one okresowej wymiany. Rozwiązaniem wymagającym najmniej ingerencji jest zastosowanie ogniw fotowoltaicznych. Podejście to jednak wymaga uzupełnienia układu o magazyn energii umożliwiający podtrzymanie pracy czujnika w okresach braku nasłonecznienia. Powszechnie stosowanym rozwiązaniem są ogniwa litowo-jonowe, których wybór uzasadnia malejący koszt, szeroka dostępność rynkowa oraz rosnąca gęstość energetyczna (ang. *energy density*) [21][TODO: Nowsze źródło]. Ich wadami są jednak stosunkowo złożony układ ładowania i zabezpieczenia przed nadmiernym rozładowaniem oraz ograniczona odporność na duże wahania temperatur [23].

Wymienionych ograniczeń pozbawione są superkondensatory. Podobnie jak ogniwa litowo-jonowe, charakteryzują się one malejącym kosztem i rosnącą dostępnością, a ponadto wykazują znacznie lepszą odporność na wahania temperatury [23] oraz nie wymagają ograniczenia głębokości rozładowania. Układ ładowania sprowadza się w ich przypadku wyłącznie do zabezpieczenia przed przekroczeniem maksymalnego napięcia na okładkach. Cechy te czynią superkondensatory rozwiązaniem szczególnie odpowiednim dla zewnętrznych czujników zasilanych fotowoltaicznie. Istotną wadą superkon-

densatorów pozostaje jednak znacznie mniejsza gęstość energetyczna w porównaniu z ogniwami litowo-jonowymi, co ogranicza ich zastosowanie do urządzeń o niskim poborze mocy.

Rozdział 2

Programowanie systemów wbudowanych

System wbudowany (ang. *embedded system*) stanowi wyspecjalizowany system komputerowy, będący integralną częścią większego układu mechanicznego lub elektronicznego. W przeciwieństwie do komputerów osobistych ogólnego przeznaczenia (ang. *general-purpose computers*), systemy te są projektowane do realizacji ściśle zdefiniowanych zadań, co pozwala na optymalizację pod kątem wydajności, kosztów oraz zużycia energii. Współcześnie systemy wbudowane znajdują zastosowanie w szerokim spektrum urządzeń — od sprzętu gospodarstwa domowego (AGD), po zaawansowane systemy sterowania w przemyśle i motoryzacji [8].

2.1 Charakterystyka układów sterujących

Kluczowym elementem decyzyjnym systemu wbudowanego jest układ scalony o wysokim stopniu integracji — mikrokontroler lub mikroprocesor. Mikroprocesor (MPU) integruje jednostkę centralną (CPU) oraz rejestry danych, jednak do poprawnej pracy wymaga zewnętrznych komponentów, takich jak pamięć operacyjna (RAM), pamięć nieulotna oraz kontrolery peryferiów. Z kolei mikrokontroler (MCU) jest układem autonomicznym, który w jednej obudowie zawiera procesor, pamięć oraz układy wejścia-wyjścia [35].

Wybór konkretnego układu zależy od specyfiki projektu. Do najpopularniejszych rodzin mikrokontrolerów należą [20] [1]:

- **Rodzina AVR (Atmel/Microchip):** 8-bitowe układy charakteryzujące się prostotą programowania, spopularyzowane dzięki platformie Arduino. Są optymalne dla prostych systemów sterowania.

- **Rodzina ESP (Espressif Systems):** Układy ESP8266 oraz ESP32, które rewolucjonizowały rynek dzięki zintegrowanym modułom łączności bezprzewodowej Wi-Fi oraz Bluetooth, przy zachowaniu bardzo korzystnego stosunku ceny do wydajności.
- **STM32 (STMicroelectronics):** 32-bitowe mikrokontrolery oparte na architekturze ARM Cortex-M, oferujące wysoką moc obliczeniową i bogaty zestaw peryferiów, powszechnie stosowane w rozwiązaniach profesjonalnych.

2.2 Języki programowania

Wybór odpowiedniego języka programowania ma kluczowe znaczenie dla efektywności procesu wytwarzania oprogramowania, stabilności systemu oraz optymalnego wykorzystania ograniczonych zasobów sprzętowych mikrokontrolera. W architekturze systemów wbudowanych tradycyjnie dominują języki niskopoziomowe, jednak rozwój technologii oraz wzrost mocy obliczeniowej układów scalonych przyczyniły się do popularyzacji alternatywnych rozwiązań.

2.2.1 Język C

Język C od dekad pozostaje standardem branżowym w dziedzinie systemów wbudowanych [9]. Jego główną zaletą jest wysoka wydajność, minimalny narzut pamięciowy oraz bezpośredni dostęp do rejestrów procesora i zasobów sprzętowych. Kod źródłowy kompilowany jest bezpośrednio do kodu maszynowego, a brak ukrytych warstw abstrakcji zapewnia wysoką przewidywalność czasu wykonania (ang. *deterministic execution time*). Ponadto, producenci mikrokontrolerów dostarczają dedykowane biblioteki HAL (ang. *Hardware Abstraction Layer*) najczęściej właśnie w języku C.

Główną wadą języka C jest brak wbudowanych mechanizmów bezpieczeństwa pamięci. Problemy takie jak przepełnienie bufora (ang. *buffer overflow*), wycieki pamięci (ang. *memory leak*) czy *use-after-free* muszą być kontrolowane bezpośrednio przez programistę, co zwiększa ryzyko wystąpienia krytycznych błędów w działaniu urządzenia [28].

2.2.2 Język C++

Język C++ wprowadza do świata systemów wbudowanych paradygmat programowania obiektowego (OOP) oraz mechanizmy abstrakcji, takie jak szablony (ang. *templates*) czy polimorfizm. Pozwala to na lepszą strukturyzację i modularność kodu, co jest szczególnie istotne w złożonych projektach [18].

W kontekście mikrokontrolerów stosuje się zazwyczaj ograniczony podzbiór nowoczesnego języka C++ [7]. Najczęściej rezygnuje się z dynamicznej alokacji pamięci (operatorów *new/delete*) oraz obsługi wyjątków (ang. *exceptions*), ponieważ mechanizmy te wprowadzają nieprzewidywalność czasu wykonania programu i mogą prowadzić do fragmentacji pamięci RAM.

2.2.3 MicroPython

MicroPython to zoptymalizowana implementacja języka Python 3, stworzona z myślą o pracy na nowoczesnych mikrokontrolerach (np. ESP32, STM32). Jego kluczową zaletą jest niezwykle wysoki poziom abstrakcji, prostota składni oraz interpretacja kodu w czasie rzeczywistym, co drastycznie skraca czas tworzenia prototypów [22].

Niestety, jako język interpretowany, MicroPython charakteryzuje się znacznie niższą wydajnością i większym zużyciem pamięci RAM oraz Flash w porównaniu do języków kompilowanych. Obecność mechanizmu odśmiecania pamięci (ang. *Garbage Collector*) wyklucza go z zastosowań w systemach twardego czasu rzeczywistego (ang. *hard real-time system*), gdzie wymagany jest rygorystyczny determinizm czasowy.

2.2.4 Język Rust

Język Rust reprezentuje nowoczesne podejście do programowania systemowego, łącząc wydajność i niskopoziomą kontrolę znaną z języków C/C++ z nowoczesnymi mechanizmami bezpieczeństwa. W kontekście systemów wbudowanych, kluczową cechą języka Rust jest koncepcja zarządzania pamięcią oparta na systemie własności (ang. *ownership*) oraz cyklach życia obiektów (ang. *lifetimes*). Walidacja tych reguł odbywa się na etapie kompilacji (ang. *compile-time*), co eliminuje całą klasę błędów związanych z dostępem do pamięci bez wprowadzania narzutu w czasie wykonywania programu oraz bez użycia mechanizmu *Garbage Collector*.

Dodatkowo, ekosystem języka Rust oferuje zaawansowane narzędzia menedżera pakietów **cargo**, co ułatwia zarządzanie zależnościami, instalowanie bibliotek oraz automatyzację testów. Choć próg wejścia w ten język jest wysoki ze względu na restrykcyjny kompilator oraz niestandardowe podejście do zarządzania pamięcią, bezpieczeństwo kodu i odporność na błędy współbieżności (ang. *data races*) sprawiają, że staje się on coraz częstszym wyborem w projektowaniu niezawodnych systemów wbudowanych [32].

2.2.5 Podsumowanie i porównanie cech języków

W celu uzasadnienia wyboru narzędzi programistycznych w niniejszej pracy, w tabeli 2.1 zestawiono najważniejsze cechy omawianych języków programowania.

Tabela 2.1: Porównanie języków programowania w systemach wbudowanych

Cecha / Kryterium	C	C++	MicroPython	Rust
Wydażność obliczeniowa	Bardzo wysoka	Bardzo wysoka	Niska	Bardzo wysoka
Zarządzanie pamięcią	Ręczne	Ręczne / Automatyczne	Garbage Collector	Statyczne (Kompilator)
Bezpieczeństwo pamięci	Brak	Ograniczone	Wysokie	Całkowite (Safe Rust)
Poziom abstrakcji	Niski	Średni / Wysoki	Bardzo wysoki	Wysoki
Determinizm czasowy	Tak	Tak (z ograniczeniami)	Nie	Tak

Rozdział 3

Realizacja projektu czujników klimatycznych

Zadaniem tego rozdziału jest opisanie rozwiązań wybranych do realizacji części praktycznej licencjatu.

3.1 Założenia projektowe

Projekt od początku powstawał z myślą zaprojektowania czujników klimatycznych do zastosowań wewnętrznych jak i zewnętrznych. Takie podejście podyktowało pewne założenia, które gotowy projekt musi spełnić. Głównymi z nich są:

- **Komunikacja bezprzewodowa** — czujnik nie powinien wymagać przewodowego połączenia ze stacją bazową.
- **Własne źródło energii** — czujnik powinien posiadać własne źródło zasilania, aby jak najbardziej zredukować czas potrzebny na konserwację czujnika.
- **Magazyn energii** — czujnik powinien być w stanie dokonywać pomiarów nawet w czasie braku dostępu do energii pochodzącej ze źródła zasilania.
- **Niski pobór mocy** — czujnik powinien potrzebować niewielkich ilości energii do pracy, aby przedłużyć czas operowania na zmagazynowanej energii i pozwolić na szybsze jego ładowanie.
- **Akwizycja podstawowych danych** — czujnik powinien zbierać podstawowe dane klimatyczne, takie jak: temperatura, wilgotność i ciśnienie.
- **Rozszerzalność** — czujnik powinien umożliwiać rozszerzenie o dodatkowe moduły pomiarowe.

3.2 Moduły pomiarowe

W celu akwizycji podstawowych danych wybrano wspomniane wcześniej, uzupełniające się moduły pomiarowe AHT20 — odpowiedzialny za pomiar temperatury i wilgotności — oraz MBP280 — mierzący ciśnienie. Dodatkowo użyto modułu VEML7700 rozszerzający zestaw pomiarowy o natężenie światła. Wszystkie komunikują się protokołem I²C co ułatwia zaprojektowanie połączeń z mikrokontrolerem — wymagają one jedynie wspólnej magistrali dwuprzewodowej — oraz pozwala na łatwe rozszerzenie układu o dodatkowe moduły pomiarowe używające tej magistrali. Dodatkowym kryterium doboru była dostępność szczegółowej dokumentacji technicznej w postaci kart katalogowych producenta, umożliwiającej samodzielną implementację sterowników programowych. Projekt został wyposażony również w licznik Geigera-Müllera w celu ostrzegania przed zwiększonym poziomem promieniowania.

3.3 Zasilanie

Aby spełniać założenie o własnym źródle energii, wybrano panele fotowoltaiczne, a w celu zapewnienia nieprzerwanej pracy czujniki zostały wyposażone w superkondensatory i układ zabezpieczający je przed przeładowaniem. Wybór superkondensatorów został uzasadniony w podrozdziale 1.4.

3.4 Mikrokontroler

Ze względu na wysoką integrację, niski koszt implementacji oraz uproszczoną architekturę sprzętową, w niniejszym projekcie zdecydowano się na wykorzystanie mikrokontrolera. Niski stopień skomplikowania pracy przyczynił się do wyboru mikrokontrolera **ATmega328p** firmy Atmel, opartego o 8-bitowy procesor z architekturą AVR firmy Atmel, posiadającego 32KB wbudowanej pamięci typu flash [6]. Najważniejszymi jego cechami, w kontekście tego projektu, są niski pobór mocy, możliwość uśpienia oraz wbudowane konwertery analogowo-cyfrowe [6]. Posiada on również sprzętowe wsparcie dla protokołu I²C oraz 3 wbudowane liczniki (ang. *Timer / Counter*).

3.5 Komunikacja radiowa

Aby zapewnić bezprzewodową komunikację modułu ze stacją bazową zdecydowano się na wybór tanich modułów radiowych, posługujących się pasmem 433 MHz.

3.6 Oprogramowanie

Z porównania języków programowania przedstawionego w podrozdziale [2.2.5](#) wynika, że język Rust stanowi optymalny kompromis pomiędzy bezkompromisową wydajnością i determinizmem cechującym język C, a nowoczesnymi mechanizmami bezpieczeństwa i wysokim poziomem abstrakcji. Z tego względu został wybrany jako główne narzędzie programistyczne w realizacji części praktycznej projektu.

Rozdział 4

Część sprzętowa

Aby utworzyć system wbudowany niezbędna jest platforma wyposażona w mikrokontroler i umożliwiająca jego oprogramowanie. Na potrzeby tego projektu zdecydowano się na utworzenie autorskiej platformy łączącej mikrokontroler, wyprowadzenia modułów oraz elementy pośrednie.

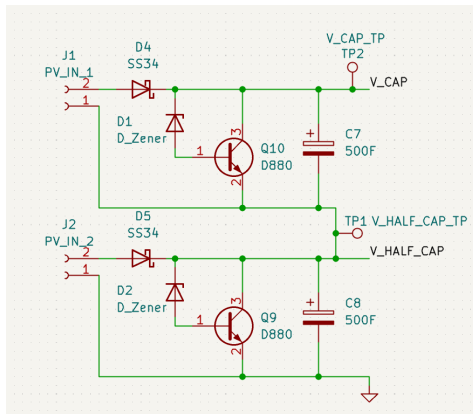
4.1 Sekcja ładowania superkondensatorów

W celu zabezpieczenia superkondensatorów przed przeładowaniem układ ładowania opiera się o regulator bocznikowy (ang. *shunt regulator*), którego zadaniem jest ograniczenie napięcia do maksymalnej wartości 2.7 V. Składa się on z tranzystora NPN i diody regulującej prąd przez niego przepływający.

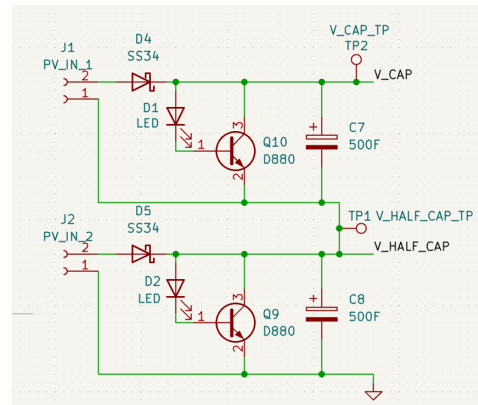
Początkowo jako diodę regulującą zastosowano diodę Zenera, wpiętą zaporowo pomiędzy linię napięcia, a bazę tranzystora (rysunek 4.1). Mimo wyboru diody, o napięciu odpowiednio pomniejszonym o spadek napięcia na złączu baza-emiter transformatora, testy wykazały, że regulowane napięcie zbyt mocno skorelowane jest z prądem emitera i istnieje możliwość, że przekroczy ono napięcie znamionowe superkondensatora.

Jako rozwiązanie tego problemu zamieniono diodę Zenera na diodę LED, włączoną w układ w kierunku przewodzenia (rysunek 4.2). Umożliwiło to znacznie lepszą kontrolę nad regulowanym napięciem. Porównanie stabilizacji napięcia dla obu obwodów przedstawiono w tabeli 4.1. Z tabeli widać, że użycie diody LED skutkuje znacznie mniejszymi różnicami napięcia na wyjściu regulatora i nie przekracza ono wartości 2.7 V.

Źródłem napięcia wejściowego dla tego układu są panele fotowoltaiczne. W celu ich ochrony przed prądem wstecznym linia napięcia dodatniego została wyposażona w diodę Schottky’ego, wybraną z racji niższego, niż w przypadku zwykłej diody, spadku napięcia.



Rysunek 4.1: Sekcja ładowania superkondensatorów z użyciem diody Zenera (D1, D2)



Rysunek 4.2: Sekcja ładowania superkondensatorów z użyciem diody LED (D1, D2)

Tabela 4.1: Napięcie wyjściowe U_{Wy} regulatora bocznikowego przy zastosowaniu diody Zenera i diody LED, dla prądu emitera I_E tranzystora zwierającego. Do testu użyto zasilacza stałoprądowego

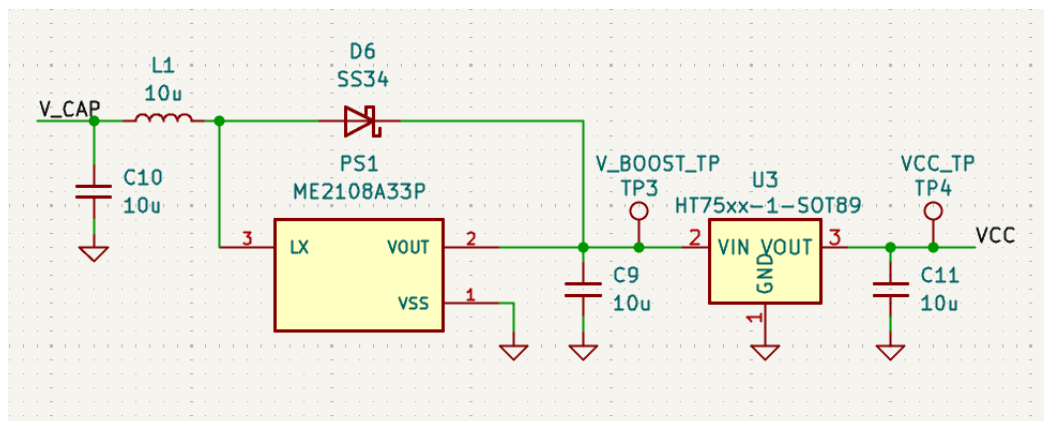
I_E [mA]	U_{Wy} Zener [V]	U_{Wy} LED [V]
5	1,80	2,38
20	2,09	2,45
50	2,25	2,50
100	2,40	2,53
200	2,65	2,57
500	3,00	2,68

4.2 Układ regulacji napięcia

Układ regulacji napięcia, przedstawiony na rysunku 4.3, składa się z 2 połączonych ze sobą części.

Pierwszą z nich jest przetwornica typu step-up, zbudowana w oparciu o układ ME210833P. Jej zastosowanie pozwala na zapewnienie zasilania gdy napięcie na połączonych szeregowo superkondensatorach spadnie poniżej 3.3 V. Pozwoli to na rozładowanie ich do łącznego napięcia około 1 V, co wynika z ograniczeń układu sterującego przetwornicy [24]. W momencie gdy napięcie wejściowe przetwornicy przekroczy 3.3 V, zostanie ona pominięta, przez diodę Schottky'ego podłączoną pomiędzy jej wejściem i wyjściem.

Napięcie jest następnie ograniczane do wartości 3.3V przy użyciu regulatora liniowego HT7333 o niskim spadku napięcia (rzędu 80 mV), i niskim prądzie spoczynkowym (o maksymalnej wartości 8 μ A) [19].



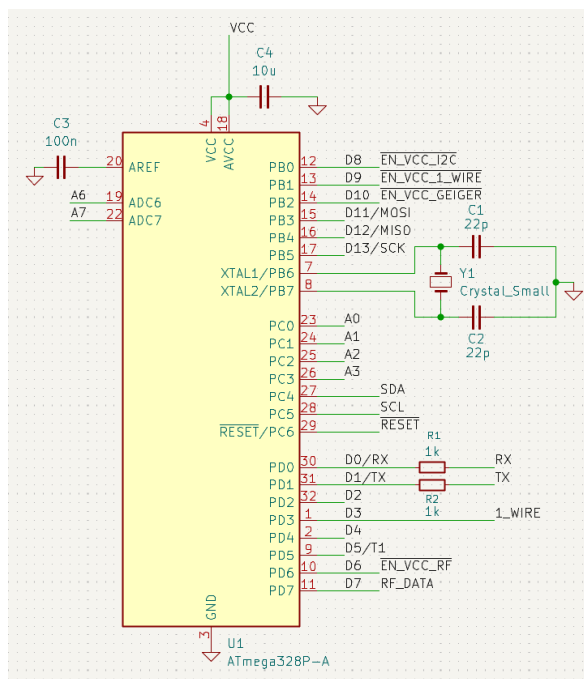
Rysunek 4.3: Układ regulacji napięcia oparty o przetwornicę step-up i regulator liniowy

4.3 Mikrokontroler ATmega328P

Jako jednostkę sterującą pracą urządzenia wybrano mikrokontroler ATmega328p rodziny AVR firmy Atmel. Został wyposażony w 8 MHz oscylator kwarcowy. Schemat połączeń mikrokontrolera widoczny jest na rysunku 4.4, a opis funkcji jego wyprowadzeń został przedstawiony w tabeli 4.2.

Tabela 4.2: Funkcje wyprowadzeń mikrokontrolera

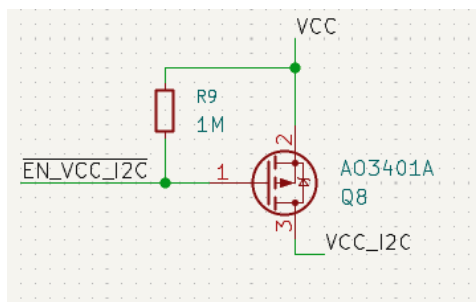
Pin	Typ pinu	Funkcja
PB0	Wyjście	Sterowanie zasilaniem urządzeń magistrali I ² C.
PC4	Dwukierunkowy	Linia danych magistrali I ² C(SDA).
PC5	Dwukierunkowy	Linia zegarowa magistrali I ² C(SCL).
PB1	Wyjście	Sterowanie zasilaniem urządzeń magistrali 1-Wire.
PD3	Dwukierunkowy	Linia danych magistrali 1-Wire.
PD6	Wyjście	Sterowanie zasilaniem nadajnika radiowego.
PD7	Dwukierunkowy	Linia danych nadajnika radiowego.
PB2	Wyjście	Sterowanie zasilaniem modułu radiometrycznego.
PD3	Wejście licznikowe	Zliczanie impulsów z modułu radiometrycznego.
PC0	ADC	Pomiar napięcia pierwszego superkondensatora.
PC1	ADC	Pomiar napięcia dwóch superkondensatorów połączonych szeregowo.



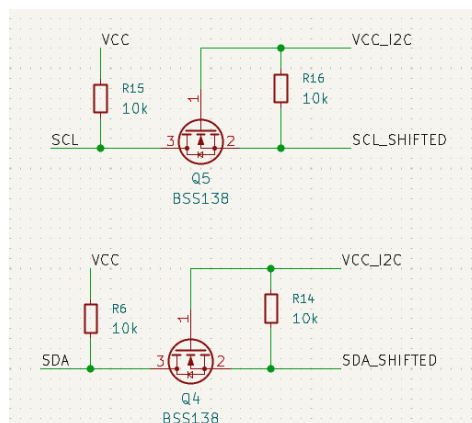
Rysunek 4.4: Mikrokontroler ATmega328P widoczny na schemacie projektu

4.4 Magistrala I²C

Obsługa magistrali realizowana jest przez dwa wyprowadzenia mikrokontrolera: PC4 i PC5. Dodatkowo użyty został pin PB0 jako pin załączający zasilanie dla urządzeń magistrali, gdy pojawi się na nim stan niski (rysunek 4.5). Aby zabezpieczyć te urządzenia przed możliwym napięciem na linii zegarowej i linii danych w momencie odłączenia linii zasilania zastosowano dla nich konwertery stanów logicznych (ang. *level shifter*), widoczne na rysunku 4.6. Jednocześnie rezystory konwertera służą za rezystory podciągające wymagane przez standard I²C [29].



Rysunek 4.5: Tranzystor załączający linię zasilania urządzeń magistrali I²C



Rysunek 4.6: Konwertery poziomów logicznych dla linii SDA i SCL magistrali I²C

4.5 Układ komunikacji radiowej

Komunikacja radiowa odbywa się poprzez bezpośrednie sterowanie linią danych modułu nadajnika radiowego podłączoną do pinu PD7 mikrokontrolera. Przygotowanie i buforowanie danych spoczywa na jednostce centralnej. Z racji dużego i ciągłego poboru mocy przez układ radiowy linia zasilania, podobnie jak w przypadku magistrali I²C, została wyposażona w odcinający przepływ prądu tranzystor, a linia danych w konwerter stanów logicznych, jak pokazano na rysunku [TODO: Wstaw rysunek].

4.6 Moduł radiometryczny

Moduł detekcji promieniowania jonizującego wykorzystuje licznik Geigera-Müllera oparty na projekcie opublikowanym na forum Elektroda [2], zmodyfikowanym na potrzeby niniejszej pracy przez inż. Karola Karpowicza (rysunek [TODO: Wstaw zdjęcie]). Układ składa się z licznika Geigera-Müllera typu J321, wzmacniacza impulsów oraz generatora wysokiego napięcia.

W chwili detekcji cząstki promieniowania na wyjściu modułu generowany jest impuls, podawany następnie na wzmacniacz zbudowany na tranzystorze NPN, widoczny na rysunku [TODO: Wstaw rysunek].

Zastosowany również został liniowy regulator napięcia 2.5 V zaopatrujący moduł w odpowiednie napięcie (rysunek [TODO: Wstaw rysunek]).

4.7 Pozostałe elementy pomiarowe

Pozostałe 3 elementy pomiarowe używają wspólnej magistrali I²C. Moduły AHT20 i BMP280 znajdują się na jednej płytce z połączonymi wyprowadzeniami więc projekt przewiduje dla nich wspólne wyjście, a moduł VEML7700 posiada oddzielne wyjście.

4.8 Pozostałe wyprowadzenia

Schemat przewiduje dodatkowe wyprowadzenia dla komunikacji SPI (*Serial Peripheral Interface*) w celu umożliwienia programowania mikrokontrolera. Dodatkowo, aby móc komunikować się za pośrednictwem portu szeregowego przewidziane zostało wyprowadzenie dla interfejsu UART.

Na schemacie znajduje się również wyjście i układ sterowania zasilaniem dla interfejsu 1-Wire, odpowiednio wyjścia PB1 i PD3. Mimo, że obecnie nie jest wspierany przez oprogramowanie, może być wykorzystany w przyszłych iteracjach projektu.

Aby zapewnić możliwość dodania dodatkowych modułów obsługujących magistralę I²C, zostało przewidziane dodatkowe jej wyprowadzenie.

Rozdział 5

Oprogramowanie

Rozdział ten opisuje oprogramowanie utworzone na potrzeby tego projektu. Omówione zostały główne elementy implementacji: obsługa licznika Geigera–Müllera, autorska biblioteka do komunikacji z prostymi nadajnikami radiowymi oraz obsługa modułów wspomnianych w podrozdziale [3.2](#).

5.1 Narzędzia i biblioteki nieautorskie

Projekt korzysta z kilku zewnętrznych bibliotek oraz narzędzi usprawniających programowanie mikrokontrolera w języku Rust.

- **avr-hal** — warstwa abstrakcji sprzętowej (*Hardware Abstraction Layer*) dla mikrokontrolerów z rodziny AVR, dostarczająca m.in. implementację **arduino-hal**, wykorzystywaną do obsługi peryferiów mikrokontrolera ATmega328p (magistrala I²C, piny GPIO, liczniki sprzętowe);
- **avr-device** — biblioteka udostępniająca niskopoziomowy dostęp do rejestrów peryferiów mikrokontrolera ATmega328p, wygenerowana na podstawie plików opisu peryferiów (SVD);
- **embedded-hal** — zbiór standardowych, niezależnych od platformy interfejsów (cech) do obsługi popularnych peryferiów (np. I²C, GPIO), umożliwiający pisanie kodu przenośnego pomiędzy różnymi mikrokontrolerami;
- **panic-halt** — minimalistyczna implementacja procedury obsługi paniki, zatrzymująca wykonywanie programu;
- **ufmt** — alternatywna dla standardowej biblioteki `core::fmt` implementacja formatowania tekstu, o znacznie mniejszym rozmiarze skompilowanego kodu, istotnym w środowisku `no_std`;

- **nb** — biblioteka dostarczająca abstrakcję operacji nieblokujących, wykorzystywaną przez **embedded-hal** oraz **avr-hal** do obsługi peryferiów pracujących asynchronicznie.

Do budowania i wgrywania oprogramowania na mikrokontroler wykorzystano dodatkowo następujące narzędzia:

- **ravedude** — narzędzie automatyzujące proces wgrywania skompilowanego programu na płytkę oraz nawiązujące komunikację z programatorem, uruchamiane automatycznie po kompilacji za pomocą polecenia **cargo run** (konfiguracja w pliku **Ravedude.toml**);
- **avrdude** — programator wykorzystywany przez **ravedude** do komunikacji z mikrokontrolerem za pośrednictwem programatora sprzętowego **usbasp**;
- niestandardowa (*unstable*) funkcjonalność narzędzia **cargo** — **build-std**. Umożliwia ona skompilowanie biblioteki **core**, niezależnego od systemu operacyjnego rdzenia biblioteki standardowej Rusta wykorzystywanego w środowiskach **no_std**, dla docelowej architektury **avr-none**, nieobecnej domyślnie w dystrybucji kompilatora Rust.

5.2 Moduł AHT20

Za obsługę modułu AHT20 odpowiedzialna jest struktura **Aht20**. Komunikacja realizowana jest za pośrednictwem magistrali I²C poprzez wysyłanie dedykowanych komend sterujących. Moduł nie wymaga przesyłania danych konfiguracyjnych w trakcie działania.

Zgodnie z notą katalogową producenta [5] urządzenie obsługuje następujący zestaw komend:

- **Inicjalizacja** — wymuszenie procedury kalibracji wewnętrznej układu;
- **Wyzwolenie pomiaru** — zainicjowanie akwizycji danych oraz ich zapis do wewnętrznej pamięci urządzenia;
- **Odczyt statusu** — weryfikacja stanu kalibracji układu oraz gotowości do przeprowadzenia kolejnego pomiaru.

Inicjalizacja

Procedura inicjalizacji modułu, przedstawiona na listingu 5.1, rozpoczyna się od odczekania czasu rozruchu wynoszącego 40 ms, po którym następuje odczyt rejestru statusu. W przypadku braku flagi kalibracji wysyłana jest komenda inicjalizacji, a następnie wprowadzane jest opóźnienie 10 ms niezbędne do jej wykonania.

Listing 5.1: Inicjalizacja modułu AHT20

```
1 pub fn init<D: DelayNs>(
2     &self, i2c: &mut I2c,
3     delay: &mut D
4 ) -> Result<(), Aht20Error> {
5     delay.delay_ms(40);
6
7     let status = self.read_status(i2c)?;
8
9     if status & STATUS_CALIBRATED_BIT == 0 {
10         i2c.write(self.address, &COMMAND_INITIALIZE)?;
11         delay.delay_ms(10);
12     }
13
14     Ok(())
15 }
```

Odczyt danych

Odczyt surowych danych pomiarowych, przedstawiony na listingu 5.2, realizowany jest w trzech etapach. W pierwszym kroku wysyłana jest komenda wyzwolenia pomiaru, po której, zgodnie z wymaganiami noty katalogowej, następuje odczekanie 80 ms do czasu jego zakończenia. Następnie odczytywany jest rejestr statusu w celu weryfikacji gotowości układu, jeżeli bit zajętości jest ustawiony, funkcja zwraca błąd `Aht20Error::SensorBusy`. W przypadku pomyślnej weryfikacji dane pomiarowe zostają zaczytane do 7-bajtowego bufora i zwrócone w postaci struktury `Aht20RawData`.

Listing 5.2: Odczyt surowych danych z modułu AHT20

```
1 pub fn read_raw<I: I2c, D: DelayNs>(
2     &self,
3     i2c: &mut I,
4     delay: &mut D,
```

```
5 ) -> Result<Aht20RawData, Aht20Error> {
6     i2c.write(self.address, &COMMAND_START_MEASUREMENT)?;
7
8     // Datasheet 5.4 step 3: wait 80 ms, then check busy bit
9     delay.delay_ms(80);
10
11     let status = self.read_status(i2c)?;
12     if status & STATUS_BUSY_BIT != 0 {
13         return Err(Aht20Error::SensorBusy);
14     }
15
16     let mut bytes = [0u8; 7];
17     i2c.read(self.address, &mut bytes)?;
18
19     Ok(Aht20RawData { bytes })
20 }
```

Konwersja danych

Konwersja surowych danych odczytanych z modułu na stopnie Celsjusza oraz wilgotność względną wyrażoną w procentach jest możliwa przy wykorzystaniu wzorów dostarczonych w nocie katalogowej [5].

Dla temperatury podano wzór:

$$T [^{\circ}\text{C}] = \frac{S_T}{2^{20}} \cdot 200 - 50 \quad (5.1)$$

gdzie:

- T — temperatura w stopniach Celsjusza,
- S_T — surowa wartość temperatury odczytana z czujnika.

Dla wilgotności względnej podano wzór:

$$RH [\%] = \frac{S_{RH}}{2^{20}} \cdot 100\% \quad (5.2)$$

gdzie:

- RH — wilgotność względna,
- S_{RH} — surowa wartość wilgotności względnej odczytana z czujnika.

Funkcje odpowiedzialne za konwersję przedstawiono na listingu 5.3. Wzory (5.1) i (5.2) zostały przekształcone w sposób niewymagający arytmetyki zmiennoprzecinkowej — dzielenie przez 2^{20} zastąpiono przesunięciem bitowym w prawo o 20 pozycji i wykonuje się ono dopiero po przemnożeniu surowej wartości o stałą. Wprowadzono dodatkowo stałą czasu kompilacji `PRECISION` dla funkcji `convert_temperature` i `convert_humidity`, pozwalającą na skalowanie wyników i ograniczenie strat wynikających z arytmetyki całkowitoliczbowej. Przekazanie wartości `PRECISION = 1000` skutkuje zwróceniem wyników z dokładnością do tysięcznych części stopnia Celsjusza lub procenta w zależności od użytej funkcji.

Listing 5.3: Konwersja surowych danych z modułu AHT20

```
1 pub fn convert_temperature<const PRECISION: i64>(raw: i64) -> i16 {
2     (((raw * 200 * PRECISION) >> 20) - 50 * PRECISION) as i16
3 }
4
5 pub fn convert_humidity<const PRECISION: i64>(raw: i64) -> i16 {
6     ((raw * 100 * PRECISION) >> 20) as i16
7 }
```

Weryfikacja integralności

Integralność odebranych danych może zostać zweryfikowana za pomocą 8-bitowej sumy kontrolnej CRC, znajdującej się w siódmym bajcie bufora odczytu. Weryfikacja realizowana jest przez funkcję `check_crc`, przedstawioną na listingu 5.4.

Listing 5.4: Sprawdzanie sumy kontrolnej danych modułu AHT20

```
1 fn check_crc(bytes: &[u8; 7]) -> bool {
2     let mut crc: u8 = 0xFF;
3     for &byte in &bytes[..6] {
4         crc ^= byte;
5         for _ in 0..8 {
6             if crc & 0x80 != 0 {
7                 crc = (crc << 1) ^ 0x31;
8             } else {
9                 crc <<= 1;
10            }
11        }
12    }
13    crc == bytes[6]
```

14 }

Odczyt pełnych danych

Implementacja struktury `Aht20` została uzupełniona o funkcję `read` — przedstawioną na listingu 5.5 — wykonującą surowy pomiar, weryfikację integralności danych oraz ich konwersję. Surowe wartości temperatury i wilgotności są 20-bitowymi liczbami całkowitymi bez znaku, które przed przetwarzaniem zostają rozszerzone do typu `i64`. Wybór tego typu podyktowany jest koniecznością uniknięcia przepełnienia podczas pośrednich operacji arytmetycznych. Należy zaznaczyć, że typ `i64` nie jest natywnie wspierany przez architekturę AVR i musi być realizowany programowo, co negatywnie wpływa na wydajność kodu. Wartości zwracane przez funkcję podane są z dokładnością do setnych części procenta i stopnia Celsjusza, opakowane wraz z flagą integralności danych w strukturę `Aht20Data`.

Listing 5.5: Odczyt danych modułu AHT20

```
1 pub fn read<D: DelayNs>(<br>2     &self, i2c: &mut I2c,<br>3     delay: &mut D<br>4 ) -> Result<Aht20Data, Aht20Error> {<br>5     let raw = self.read_raw(i2c, delay)?;<br>6<br>7     let crc_passed = Self::check_crc(&raw.bytes);<br>8<br>9     // Humidity: bits [39:20] of the data payload (bytes 1-3)<br>10    let raw_humidity = 0i64<br>11        | ((raw.bytes[1] as i64) << 12)<br>12        | ((raw.bytes[2] as i64) << 4)<br>13        | ((raw.bytes[3] as i64) >> 4);<br>14<br>15    // Temperature: bits [19:0] of the data payload (bytes 3-5)<br>16    let raw_temperature = 0i64<br>17        | ((raw.bytes[3] as i64 & 0x0F) << 16)<br>18        | ((raw.bytes[4] as i64) << 8)<br>19        | (raw.bytes[5] as i64);<br>20<br>21    Ok(Aht20Data {<br>22        temperature:
```

```

23         Self::convert_temperature::<100>(raw_temperature),
24         humidity:
25         Self::convert_humidity::<100>(raw_humidity),
26         crc_passed,
27     })
28 }

```

5.3 Moduł BMP280

Moduł BMP280 jest obsługiwany przez strukturę `Bmp280`, pokazaną na listingu 5.6. Zawiera ona pole adresu urządzenia magistrali I²C, pole przechowujące dane kompensacyjne pomiarów oraz parametr typu `CONFIG` powiązany z polem `PhantomData` o zerowym rozmiarze w pamięci.

Listing 5.6: Struktura `Bmp280`

```

1 pub struct Bmp280<CONFIG>{
2     address: u8,
3     compensations: Option<Bmp280Compensations>,
4     _config: PhantomData<CONFIG>,
5 }

```

Dane kompensacyjne

Do poprawnej interpretacji danych modułu BMP280 niezbędne są dane kompensacyjne zapisane przez producenta w pamięci samego modułu. Składają się one z dwunastu liczb szesnastobitowych (dwie z nich traktowane są jako liczby ze znakiem) zapisanych w formacie *little endian*.

Przechowaniem ich po stronie programu zajmuje się struktura `Bmp280Compensations`. Została ona opatrzona metodą pozwalającą na ich odczyt z modułu, przedstawioną na listingu 5.7. Metoda odczytuje 24 bajty z pamięci modułu BMP280 i przypisuje je do poszczególnych pól struktury. Nazwy pól struktury przyjęto zgodnie z nazwami prezentowanymi w nocie katalogowej [15].

Listing 5.7: Odczyt danych kompensacyjnych dla modułu BMP280

```

1 pub fn read_from_i2c(address: u8, i2c: &mut I2c)
2     -> Result<Self, i2c::Error>{
3
4     let mut buffer = [0u8; N_COMPENSATION_BYTES];

```

```
5
6     i2c.write_read(
7         address,
8         &[COMPENSATION_START_ADDRESS],
9         &mut buffer
10    )?;
11
12    Ok(Self{
13        dig_t1: u16::from_le_bytes([buffer[0], buffer[1]]),
14        dig_t2: i16::from_le_bytes([buffer[2], buffer[3]]),
15        dig_t3: i16::from_le_bytes([buffer[4], buffer[5]]),
16
17        dig_p1: u16::from_le_bytes([buffer[6], buffer[7]]),
18        // ...
19        dig_p9: i16::from_le_bytes([buffer[22], buffer[23]]),
20    })
21 }
```

Konfiguracja

Konfiguracja modułu BMP280 odbywa się poprzez zapis 8-bitowych wartości do rejestrów CTRL_MEAS i CONFIG. Wartości te pozyskiwane są z typu implementującego cechę `Config`, widoczną na listingu 5.8. Cecha ta definiuje skojarzone typy odpowiedzialne za poszczególne ustawienia modułu:

- `TempOversampling` – określa stopień nadpróbkowania pomiaru temperatury,
- `PressOversampling` – określa stopień nadpróbkowania pomiaru ciśnienia,
- `PowerMode` – określa tryb pracy urządzenia,
- `Standby` – określa odstęp czasowy pomiędzy pomiarami,
- `IIR` – określa stałą filtra IIR.

Każdy z tych typów ograniczony jest do odpowiedniej cechy udostępniającej stałą `BITS`, reprezentującą wartość bitową zgodną z notą katalogową. Wymagane wartości rejestrów przechowywane są w stałych `CTRL_MEAS_VALUE` i `CONFIG_VALUE`, których wartości powstają przez przesunięcie bitowe stałych `BITS` na odpowiednie pozycje i złożenie ich operatorem alternatywy bitowej. Rejestr `CONFIG_VALUE` dodatkowo określa

protokół komunikacyjny urządzenia — pole `EN_SPI` ustawione jest na 0, co wymusza użycie protokołu I²C.

Listing 5.8: Cecha `Config` używana do konfiguracji BMP280

```

1 pub trait Config{
2     type TempOversampling: Oversampling;
3     type PressOversampling: Oversampling;
4     type PowerMode: PowerMode;
5     type Standby: Standby;
6     type IIR: IIRConstant;
7
8     const CTRL_MEAS_VALUE: u8 =
9         Self::TempOversampling::BITS << TEMP_OVERSAMPLING_OFFSET
10        | Self::PressOversampling::BITS << PRESS_OVERSAMPLING_OFFSET
11        | Self::PowerMode::BITS << POWER_MODE_OFFSET;
12
13     const CONFIG_VALUE: u8 =
14         Self::Standby::BITS << STANDBY_OFFSET
15        | Self::IIR::BITS << IIR_CONSTANT_OFFSET
16        // Disable SPI mode
17        | 0b0 << EN_SPI_OFFSET;
18 }
19
20 pub trait Oversampling{ const BITS: u8; }
21 pub trait PowerMode{ const BITS: u8; }
22 pub trait Standby{ const BITS: u8; }
23 pub trait IIRConstant{ const BITS: u8; }

```

Przykładową implementację cechy `Config` oraz cech odpowiedzialnych za poszczególne wartości bitowe przedstawiono na listingu 5.9. Definicje pomocniczych cech `Oversampling`, `PowerMode`, `Standby` i `IIRConstant` zostały pominięte, ponieważ zawierają się w listingu 5.8.

Listing 5.9: Przykładowa implementacja cech konfiguracyjnych modułu BMP280

```

1 pub struct DefaultConfig;
2
3 impl Config for DefaultConfig{
4     type TempOversampling = OversamplingX1;
5     type PressOversampling = OversamplingX2;

```

```

6     type PowerMode = PowerModeNormal;
7
8     type Standby = Standby0_5ms;
9     type IIR = IIROff;
10 }
11
12 pub struct OversamplingX1;
13 pub struct OversamplingX2;
14 impl Oversampling for OversamplingX1{ const BITS: u8 = 0b001; }
15 impl Oversampling for OversamplingX2{ const BITS: u8 = 0b010; }
16
17 pub struct PowerModeNormal;
18 impl PowerMode for PowerModeNormal{ const BITS: u8 = 0b11; }
19
20 pub struct Standby0_5ms;
21 impl Standby for Standby0_5ms{ const BITS: u8 = 0b000; }
22
23 pub struct IIROff;
24 impl IIRConstant for IIROff{ const BITS: u8 = 0b000; }

```

Inicjalizacja

Proces inicjalizacji modułu, przedstawiony na listingu 5.10, odbywa się w 3 krokach. Na początku wysyłane są dane do rejestru *CTRL_MEAS*, odpowiedzialnego za konfigurację pomiarów. Następnie uzupełniany jest rejestr *CONFIG*, regulujący pracę modułu. Na koniec, po odczekaniu 2 ms wymaganych na zapis danych, odczytywane są dane kompensacyjne.

Listing 5.10: Inicjalizacja modułu BMP280

```

1 pub fn init<D: DelayNs>(&mut self, i2c: &mut I2c, delay: &mut D)
2 -> Result<(), i2c::Error>{
3     i2c.write(
4         self.address,
5         &[CTRL_MEAS_REGISTER_ADDRESS, Self::CTRL_MEAS_VALUE]
6     )?;
7     i2c.write(
8         self.address,
9         &[CONFIG_REGISTER_ADDRESS, Self::CONFIG_VALUE]

```

```

10     )?;
11
12     delay.delay_ms(2);
13
14     self.compensations = Some(
15         Bmp280Compensations::read_from_i2c(self.address, i2c)?
16     );
17
18     Ok(())
19 }

```

Odczyt danych

Odczyt surowych danych z modułu odbywa się poprzez odczytanie wartości z pamięci modułu poczynając od pierwszego bajtu kodującego wartość ciśnienia. Wartość ciśnienia i temperatury zapisana jest na 6 bajtach, a same wartości zapisane są jako liczby 20-bitowe, wprowadza to wymóg konwersji typu liczbowego na obsługiwany przez język typ `i32`. Proces ten pokazano na listingu 5.11.

Listing 5.11: Odczyt surowych danych z modułu BMP280

```

1  pub fn read_raw(&self, i2c: &mut I2c)
2  -> Result<Bmp280RawData, i2c::Error>{
3      let mut buffer = [0u8; 6];
4
5      i2c.write_read(
6          self.address,
7          &[PRESSURE_MS_BYTE_ADDRESS],
8          &mut buffer
9      )?;
10
11     // Raw temperature and pressure are both 20 bit values
12     Ok(Bmp280RawData {
13         temperature:
14             ((buffer[3] as i32) << 12)
15             | ((buffer[4] as i32) << 4)
16             | ((buffer[5] as i32) >> 4),
17         pressure:
18             ((buffer[0] as i32) << 12)

```

```

19         | ((buffer[1] as i32) << 4)
20         | ((buffer[2] as i32) >> 4),
21     })
22 }

```

Konwersja danych

Proces konwersji surowych wartości ciśnienia i temperatury na wartości rzeczywiste jest dokładnie opisany w nocie katalogowej BMP280 [15], gdzie podano funkcję zapisane w języku C. Funkcje konwersji napisane na potrzeby tej pracy są ich odzwierciedleniem w języku Rust. Warto jednak zaznaczyć, że funkcja `convert_raw_temp` odpowiedzialna za konwersję temperatury, w odróżnieniu od funkcji podanej w nocie katalogowej, zwraca krotkę dwóch liczb, gdzie pierwszą jest przekonwertowana wartość temperatury, a drugą liczbą `t_fine` potrzebna do konwersji wartości ciśnienia.

Odczyt pełnych danych

Implementacja struktury `Bmp280` upublicznia funkcję `read` łączącą odczyt i konwersję danych, pokazano ją w listingu 5.12.

Listing 5.12: Funkcja `read` struktury `Bmp280`

```

1 pub fn read(&self, i2c: &mut I2c)
2 -> Result<Bmp280Data, Bmp280ReadError>{
3     let raw = self.read_raw(i2c)?;
4
5     let (temperature, fine_temp) =
6         self.convert_raw_temp(raw.temperature)?;
7
8     let pressure = self.convert_raw_pressure(raw.pressure, fine_temp)?;
9
10    Ok(Bmp280Data { temperature, pressure })
11 }

```

5.4 Moduł VEML7700

Moduł VEML7700 obsługiwany jest przez strukturę `Veml7700`, pokazaną na listingu 5.13. Struktura ta posiada parametr typu `CONFIG` związany z polem typu `Phan-`

tomData — strukturą, która nie zajmuje pamięci — oraz pole **address** przechowujące adres urządzenia magistrali I²C.

Listing 5.13: Struktura Veml7700

```

1 pub struct Veml7700<CONFIG>{
2     address: u8,
3     _config: PhantomData<CONFIG>,
4 }

```

Konfiguracja

Konfiguracja modułu VEML7700 odbywa się poprzez zapis danych do dwóch dwubajtowych rejestrów o adresach 0x0 i 0x3. Dodatkowo moduł oferuje możliwość wykrycia przekroczenia konkretnych wartości górnych i dolnych (ang. *thresholds*), co skutkuje ustawieniem odpowiedniego bitu w pamięci modułu. Do ustawienia tych wartości służą odpowiednio dwubajtowe rejestry o adresach 0x1 i 0x2.

W celu przedstawienia konfiguracji została utworzona cecha **Config** oraz cechy pomocnicze przedstawione na listingu 5.14. Głównym zadaniem tej cechy jest określenie wartości rejestrów bazując na stałych czasu kompilacji **BITS** dostępnych w cechach pomocniczych.

Rejestry progowe mają zawsze wartość 0, ponieważ ta funkcjonalność nie jest wykorzystywana w tym projekcie.

Z racji wpływu konfiguracji na stałe matematyczne używane przy konwersji surowych wartości odczytanych z modułu, cecha została uzupełniona o stałe **LUX_NUM** i **LUX_DEN** używane w późniejszych obliczeniach. Ich wartości można obliczyć ze wzorów:

$$LUX_DEN = Sensitivity \cdot Scale \cdot T_{refresh}, \quad (5.3)$$

gdzie:

- *Sensitivity* — wartość czułości z tabeli „REFRESH TIME, IDD, AND SENSITIVITY RELATION” obecnej w nocie katalogowej [36];
- *T_{refresh}* — wartość czasu odświeżania z tabeli „REFRESH TIME, IDD, AND SENSITIVITY RELATION” obecnej w nocie katalogowej [36];
- *Scale* — wartość dobrana w zależności od wymaganej dokładności pomiaru, mająca za zadanie zmniejszyć błąd wynikający z zastosowania arytmetyki całkowitoliczbowej.

$$LUX_NUM = 10 \cdot Scale, \quad (5.4)$$

gdzie:

- *Scale* — wartość identyczna do tej, użytej przy obliczaniu LUX_DEN.

Listing 5.14: Cecha Config dla VEML7700

```

1 pub trait Config{
2     type Sm: AlsSm;
3     type It: AlsIt;
4     type Psm: AlsPsm;
5
6     // Values of registers
7     const BITS_0x00: u16 = Self::Sm::BITS << SM_OFFSET
8                               | Self::It::BITS << IT_OFFSET;
9     const BITS_0x03: u16 = Self::Psm::BITS << PSM_OFFSET
10                               | 0b1 << PSM_EN_OFFSET;
11
12     // Registers 0x01 and 0x02 are unused as they code for thresholds
13     const BITS_0x01: u16 = 0;
14     const BITS_0x02: u16 = 0;
15
16     /// Lux calculation numerator
17     const LUX_NUM: u32;
18     /// Lux calculation denominator
19     const LUX_DEN: u32;
20 }
21
22 /// Sensitivity mode
23 pub trait AlsSm{ const BITS: u16; }
24
25 /// Integration time
26 pub trait AlsIt{ const BITS: u16; }
27
28 /// Power saving mode
29 pub trait AlsPsm{ const BITS: u16; }

```

Przykładowa konfiguracja, używana na potrzeby tego projektu, jest pokazana na listingu 5.15. Dane pochodzą z noty katalogowej modułu [36].

Listing 5.15: Przykładowa konfiguracja dla modułu VEML7700

```

1  pub struct ConfigFastLowPower;
2
3  impl Config for ConfigFastLowPower{
4      type It = AlsIt25ms;
5      type Sm = AlsSm_x2;
6      type Psm = AlsPsm1;
7
8      const LUX_NUM: u32 = 10 * 1000;
9      // Sensitivity of 0.042 scaled by 1000
10     // and refresh time of 600ms
11     const LUX_DEN: u32 = 42 * 600;
12 }
13
14 pub struct AlsIt25ms;
15 impl AlsIt for AlsIt25ms{ const BITS: u16 = 0b1100; }
16
17 pub struct AlsSm_x2;
18 impl AlsSm for AlsSm_x2{ const BITS: u16 = 0b01; }
19
20 pub struct AlsPsm1;
21 impl AlsPsm for AlsPsm1{ const BITS: u16 = 0b00; }

```

Inicjalizacja

Inicjalizacja modułu VEML7700 sprowadza się do wpisania danych do 4 rejestrów konfiguracyjnych. W tym celu implementacja struktury `Veml7700` posiada stałą czasu kompilacji będącą tablicą wartości, które powinny przyjąć rejestry. Funkcja `init` iteruje przez te wartości, wpisując je do rejestrów. Pomiedzy każdym zapisem danych występuje przerwa co najmniej 10 ms. Proces ten został przedstawiony na listingu 5.16.

Listing 5.16: Funkcja `init` struktury `Veml7700`

```

1  impl<CONFIG: Config> Veml7700<CONFIG>{
2      const REGISTERS: [u16; 4] = [
3          CONFIG::BITS_0x00,
4          CONFIG::BITS_0x01,
5          CONFIG::BITS_0x02,
6          CONFIG::BITS_0x03,

```

```

7     ];
8
9     // [...]
10
11     pub fn init(&self, i2c: &mut I2c)
12     -> Result<(), arduino_hal::i2c::Error>{
13         for i in 0..Self::REGISTERS.len()
14         {
15             let value = Self::REGISTERS[i];
16             let lsb = (value & 0xFF) as u8;
17             let msb = (value >> 8) as u8;
18
19             i2c.write(self.address, &[i as u8, lsb, msb])?;
20             // Add delay for write
21             Delay::<MHz8>::new().delay_ms(10);
22         }
23
24         Ok(())
25     }
26
27     // [...]
28 }

```

Konwersja danych

Wzór matematyczny służący do konwersji surowych danych pobranych z modułu na wartość wyrażoną w luksach znajduje się w nocie katalogowej [36]. Na potrzeby kodu został on przekształcony na $lux = \frac{raw \cdot LUX_NUM}{LUX_DEN}$. LUX_DEN i LUX_NUM zostały opisane w podrozdziale 5.4. Implementacja tego wzoru widoczna jest na listingu 5.17.

Listing 5.17: Konwersja surowych danych modułu VEML7700 na wartość wyrażoną w luksach

```

1 fn raw_to_lux(raw: u16) -> u32{
2     (raw as u32 * Self::LUX_NUM) / Self::LUX_DEN
3 }

```

Odczyt danych

Odczyt danych z modułu polega na odczytaniu wartości rejestru 0x5, wewnątrz którego zapisana jest 16-bitowa liczba bez znaku. Liczba ta następnie poddawana jest konwersji. Metoda realizująca ten proces została pokazana na listingu 5.18.

Listing 5.18: Odczyt danych z modułu VEML7700

```
1 pub fn read(&self, i2c: &mut I2c)
2 -> Result<u32, arduino_hal::i2c::Error>{
3     let mut buffer = [0u8; 2];
4
5     i2c.write_read(
6         self.address,
7         &[VALUE_REGISTER_INDEX],
8         &mut buffer
9     )?;
10
11     let raw = u16::from_le_bytes([buffer[0], buffer[1]]);
12
13     Ok(Self::raw_to_lux(raw))
14 }
```

5.5 Moduł z licznikiem Geigera-Müllera

Obsługa licznika odbywa się za pośrednictwem struktury `GeigerCounter`. Posiada ona 60-elementowy bufor 16-bitowych liczb całkowitych, 32-bitową sumę wartości bufora, referencję do licznika T1 oraz pola pomocnicze. Cała struktura została przedstawiona na listingu 5.19.

Listing 5.19: Struktura `GeigerCounter`

```
1 pub struct GeigerCounter<'a>{
2     counts: [u16; 60],
3     index: usize,
4     sum: u32,
5     filled: u8,
6     timer: &'a TC1,
7 }
```

Konfiguracja licznika

Sprzętowy licznik T1 mikrokontrolera skonfigurowany jest do zliczania impulsów zewnętrznych generowanych przez moduł radiometryczny. Użyte dane konfiguracyjne pochodzą z dokumentacji mikrokontrolera ATmega328p [6]. Metoda konfiguracyjna została przedstawiona na listingu 5.20 i sprowadza się do zapisania konfiguracji do 2 rejestrów kontrolnych licznika T1: *TCCR1A* i *TCCR1B*.

Listing 5.20: Metoda init struktury GeigerCounter

```
1  /// Sets up 'self.timer' as an external pulse counter
2  pub fn init(&self){
3      let tc = self.timer;
4
5      unsafe {
6          tc.tccr1a().write(|w| w.wgm1().bits(0b00));
7
8          tc.tccr1b().write(|w| {
9              w.wgm1().bits(0b00)
10             .cs1().bits(0b111)
11         });
12
13         tc.tcnt1().write(|w| w.bits(0));
14     }
15 }
```

Odczyt licznika

Struktura *GeigerCounter* udostępnia metodę *tick*, która odczytuje 16-bitową wartość licznika T1 (rejstry *TCNT1L* i *TCNT1H*), zeruje go, a następnie zapisuje odczytaną wartość do bufora cyklicznego, wypełnionego początkowo zerami. Docelowo metoda *tick* ma być wywoływana co sekundę, dzięki czemu zapelnienie bufora zajmie minutę. Listing 5.21 przedstawia metodę *tick* oraz pomocniczą, prywatną metodę *read_and_reset_timer*, odpowiedzialną za odczyt i wyzerowanie wartości licznika T1.

Listing 5.21: Metoda init struktury GeigerCounter

```
1  pub fn tick(&mut self){
2      self.sum -= self.counts[self.index] as u32;
3
4      let count = self.read_and_reset_timer();
```

```
5
6     self.counts[self.index] = count;
7     self.sum += count as u32;
8
9     // Advance index, wrapping at 60
10    self.index = if self.index == 59 { 0 } else { self.index + 1 };
11
12    if self.filled < 60 {
13        self.filled += 1;
14    }
15 }
16
17 fn read_and_reset_timer(&self) -> u16{
18     let tc = self.timer;
19
20     avr_device::interrupt::free(|_| unsafe {
21         let val = tc.tcmt1().read().bits();
22         tc.tcmt1().write(|w| w.bits(0));
23         val
24     })
25 }
```

Odczyt zliczeń na minutę

Zliczenia na minutę (ang. *counts per minute*, CPM) określają liczbę cząstek zliczonych przez licznik promieniowania w ciągu minuty. Aby uzyskać tę wartość struktura `GeigerCounter` udostępnia metodę `cpm` pokazaną na listingu 5.22. Zwraca ona wartość poprzez ekstrapolację zsumowanych wartości bufora na czas pełnej minuty. Podejście to niesie ze sobą dwie konsekwencje: przy czasie pracy krótszym niż minuta wynik obarczony jest błędem, natomiast przy pełnym buforze uśredniane są nagłe skoki w ilości zliczeń.

Listing 5.22: Metoda `cpm` struktury `GeigerCounter`

```
1 /// Counts per minute
2 pub fn cpm(&self) -> u32{
3     // Scale to 60 seconds even before buffer is full
4     self.sum * 60 / self.filled as u32
5 }
```

5.6 Obsługa modułu radiowego

Obsługa modułu radiowego realizowana jest poprzez autorską bibliotekę `ook_433mhz` realizującą przesył i odczyt danych z prostych nadajników radiowych używając metody *On-Off Keying* (OOK). Biblioteka jest mocno inspirowana biblioteką `RadioHead`, ze szczególnym naciskiem na zawartą w niej klasę `RH_ASK`.

Ten podrozdział jest poświęcony obsłudze nadajnika radiowego (struktura `Transmitter` biblioteki `ook_433mhz`) i nie będzie zagłębiał się w implementację obsługi odbiornika.

5.6.1 Przebieg transmisji

Transmisję danych można podzielić na 3 etapy: synchronizację, wysłanie bajtu startu i wysłanie ramki danych. Sygnał synchronizujący składa się z 3 bajtów wypełnionych naprzemiennie bitami o wartościach 1 i 0. Służy on do przygotowania odbiornika na odczyt danych. Bajt startu (zdefiniowany w bibliotece stałą `START_BYTE`) przyjmuje formę liczby o zapisie binarnym `11001100`. Struktura ramki obejmuje pole długości, zawierające informację o liczbie przesyłanych bajtów, oraz następujące po nim dane właściwe.

Dane transmitowane są poczynając od najmłodszego bitu każdego bajtu. Same bity podzielone są na próbki (nazywane w programie angielskim słowem *tick*) używane do sterowania linią danych nadajnika radiowego. Każda próbka przyjmuje wartość 0 lub 1, identyczną z wartością bitu, który buduje.

5.6.2 Maszyna stanów

Struktura `Transmitter` posiada pole `state` określające obecny stan nadajnika, wybrany spośród pięciu wartości opisanych w typie wyliczeniowym `TxState`:

1. `Idle` — stan ustawiany jako domyślny i osiągany w momencie zakończenia transmisji.
2. `Syncing` — stan, podczas którego wysyłane są bajty sygnału synchronizującego, przypisywany nadajnikowi w momencie, gdy dane w buforze są gotowe do wysłania.
3. `SendingStartByte` — stan osiągany po wysłaniu wszystkich wymaganych bajtów synchronizacyjnych, w którym nadajnik wysyła bajt startu.
4. `SendingData` — stan, w którym nadajnik wysyła zawartość bufora, osiągany po wysłaniu bajtu startu.

5. `DataSent` — stan osiągany po wysłaniu całej zawartości bufora.

5.6.3 Struktura Transmitter

Struktura `Transmitter`, pokazana na listingu 5.23, przechowuje informacje niezbędne do prawidłowego nadania danych i określenia obecnego stanu nadajnika.

Głównym zadaniem pól `bit_index` oraz `byte_index` jest określenie, który bajt i zawarty w nim bit powinien być w danym momencie wysyłany.

Pole `buffer` przechowuje zakodowane dane przeznaczone do wysłania. Jego wielkość ograniczona jest przez stałą czasu kompilacji `MAX_BUFFER_SIZE`. Bufor inicjalizowany jest wartościami zerowymi. Podczas pracy nadajnika nie jest on czyszczony — jedynie nadpisywany nowymi wartościami, których długość określa pole `message_bit_length`.

Struktura posiada również parametr typu generycznego `TICKS_PER_BIT` określający liczbę próbek potrzebnych na przesłanie jednego bitu. Parametr ten określany jest w czasie kompilacji.

Listing 5.23: Struktura `Transmitter`

```
1 pub struct Transmitter<const TICKS_PER_BIT: u8, Pin: OutputPin> {
2     buffer: [u8; MAX_BUFFER_SIZE],
3     message_bit_length: usize,
4     bit_index: usize,
5     byte_index: usize,
6     state: TxState,
7     pub pin: Pin,
8     ticks: u8,
9 }
```

5.6.4 Przygotowanie danych, kodowanie 4b6b

Kodowanie 4B6B (ang. *4-bit/6-bit*) to schemat kodowania liniowego, w którym każdej 4-bitowej grupie danych (16 możliwych kombinacji) przypisywana jest odpowiadająca jej 6-bitowa grupa kodowa (spośród 64 możliwych kombinacji). Dodanie dwóch nadmiarowych bitów pozwala na taki dobór sekwencji wyjściowych, aby spełniały one dwa warunki: ograniczały maksymalną liczbę kolejnych jedynek i zer w strumieniu (tzw. kodowanie RLL, ang. *run-length limited*) oraz zapewniały ich zbalansowany udział (ang. *DC balance*). Dzięki temu przesyłany sygnał zawiera regularne przejścia między stanami, co ułatwia odbiornikowi odtworzenie zegara transmisji (ang. *clock*

recovery) bezpośrednio z odbieranego sygnału, bez potrzeby przesyłania osobnego sygnału taktującego.

Za przygotowanie danych do nadania odpowiada metoda `send` struktury `Transmitter`, przedstawiona na listingu 5.24. Sprawdza ona czy bajty wysyłanej wiadomości, po zakodowaniu, zmieszczą się w buforze. W przypadku, gdy wiadomość jest zbyt długa, zostaje ona przycięta do maksymalnej możliwej długości (określonej stałą `MAX_MESSAGE_LENGTH`). Następnie jako pierwszy bajt bufora zapisywany jest rozmiar wiadomości w bajtach przed kodowaniem, a kolejne bajty bufora uzupełnione są zawartością wiadomości. Tak przygotowany bufor zawsze będzie przynajmniej w połowie niewykorzystany, co pozwala na zakodowanie jego zawartości metodą `encode_in_place` modułu `data_coding::radio_head_4b6b`. Liczba bitów zakodowanej ramki, gotowej do wysłania przez nadajnik, określa się sumując liczbę bajtów niezakodowanej wiadomości z ilością bajtów kodujących jej rozmiar. Otrzymaną wartość mnoży się następnie przez stałą określającą narzut kodowania, wyrażoną jako liczba bitów na bajt. Funkcja zwraca liczbę bajtów zapisanych do bufora, co pozwala przysyłać większe porcje danych podzielone na osobno wysyłane fragmenty.

Listing 5.24: Metoda `send` struktury `Transmitter`

```
1 pub fn send(&mut self, bytes: &[u8]) -> Option<usize> {
2     let n_bytes = if bytes.len() > MAX_MESSAGE_LENGTH {
3         MAX_MESSAGE_LENGTH
4     } else {
5         bytes.len()
6     };
7
8     // Set message length as first byte
9     self.buffer[0] = n_bytes as u8;
10    // Populate buffer
11    self.buffer[1..n_bytes].copy_from_slice(&bytes[0..n_bytes]);
12
13    self.message_bit_length = (n_bytes + 1) * 6 * 2;
14
15    // Encode data
16    match encode_in_place(&mut self.buffer, n_bytes + 1) {
17        Ok(_) => {
18            self.state = TxState::Syncing;
19            Some(n_bytes)
20        }
```



```

21         Err(_) => None,
22     }
23 }

```

5.6.5 Obsługa maszyny stanów

Struktura `Transmitter` udostępnia metodę `transmit` odpowiedzialną za przechodzenie pomiędzy stanami, wywoływanie funkcji pośrednich obsługujących poszczególne stany i podtrzymanie stanu pinu na długość konieczną do wysłania wszystkich próbek dla jednego bitu. Funkcje pośrednie zwracają jeden z trzech stanów, informując metodę `transmit` o etapie wykonania swoich czynności lub o wystąpieniu błędu.

Wywołanie metody `transmit` powinno zachodzić w równych interwałach. W środowisku systemów wbudowanych dobrym sposobem osiągnięcia tego jest zastosowanie przerwań pochodzących z liczników mikrokontrolera.

5.7 Serializacja danych

Aby przesłać dane za pomocą sygnału cyfrowego, niezbędne jest przekształcenie ich na zapis bitowy.

Dane pomiarowe

Do interpretacji danych pomiarowych niezbędne są 3 informacje:

- **typ danych** (SENSOR TYPE) — z jakiego typu urządzenia pomiarowego pochodzą dane (np. termometr, barometr itp.)
- **numer urządzenia pomiarowego** (ID) — czujnik klimatyczny może posiadać wiele modułów pomiarowych tego samego typu.
- **rozdzielczość danych** (SCALE) — ilość liczb po przecinku zawartych w pomiarze.

Ze względu na chęć zachowania jak najmniejszego rozmiaru, informacje te znajdują się w jednym bajcie poprzedzającym samą odczytaną wartość. Bajt ten posiada następującą strukturę:

7	6	5	4	3	0
ID		SCALE		SENSOR TYPE	

Wartość pomiaru zapisywana jest jako 32 bitowa liczba bez znaku. W przypadku danych, w których występują liczby ujemne (np. temperatura), konieczna jest konwersja na liczbę bez znaku podczas jej serializacji i następnie konwersja na liczbę ze znakiem przy deserializacji. Typ ten został wybrany z powodu możliwości zapisania w nim danych z każdego obsługiwanego w projekcie modułu.

Całość wartości pomiarowej wraz z danymi ją opisującymi ma rozmiar 5 bajtów i strukturę:



Serializacja w kodzie

Za serializację danych pomiarowych odpowiedzialna jest struktura `TypedLabelReadout`, przedstawiona na listingu 5.25. Posiada ona cztery bajtowe pole danych typu `u32` oraz trzy parametry generyczne `ID`, `SCALE` i `TYPE`, odpowiedzialne odpowiednio za: numer urządzenia pomiarowego, rozdzielczość danych i typ urządzenia pomiarowego.

Listing 5.25: Struktura `Transmitter`

```

1 pub struct TypedLabelReadout<ID, SCALE, TYPE>{
2     data: u32,
3     _label: PhantomData<(ID, SCALE, TYPE)>,
4 }
```

Implementacja struktury zawęza te parametry do określonych typów, widocznych na listingu 5.26. Typy te posiadają w swojej implementacji skojarzone stałe `BITS` będące częściami bajtu informacji o pomiarze. Przy pomocy stałych opisujących przesunięcia i maski, dla każdej z informacji, utworzona jest stała `LABEL`.

Listing 5.26: Implementacja struktury `TypedLabelReadout`

```

1 impl<ID: SensorId, SCALE: UnitScale, TYPE: SensorType> TypedLabelReadout<ID, SCALE,
2     const LABEL: u8 =
3         (ID::BITS << SENSOR_ID_OFFSET) & SENSOR_ID_MASK
4         | (SCALE::BITS << UNIT_SCALE_OFFSET) & UNIT_SCALE_MASK
5         | (TYPE::BITS << SENSOR_TYPE_OFFSET) & SENSOR_TYPE_MASK;
6
7     // [...]
8 }
```

Aby umożliwić tworzenie innych struktur mogących reprezentować dane pomiarowe utworzono cechę `LabeledReadout`, pokazaną na listingu 5.27. Definiuje ona trzy metody pozwalające na uzyskanie danych odnośnie pomiaru:

- `get_label` — informacje dotyczące pomiaru,
- `get_data` — dane pomiarowe,
- `get_bytes` — dane wraz z informacjami w postaci tablicy bajtów.

Cecha posiada wewnętrzny typ `Data` określający jakiego typu dane znajdują się w jej implementacji. Z tego względu implementacja posiada również stały parametr `N_BYTES` określający ilość bajtów zwracanych przez metodę `get_bytes`, której wartość definiowana jest jako ilość bajtów typu danych zsumowana z ilością bajtów potrzebnych na zakodowanie informacji o pomiarze.

Listing 5.27: Struktura `Transmitter`

```
1 pub trait LabeledReadout<const N_BYTES: usize>{
2     type Data;
3
4     fn get_label(&self) -> u8;
5     fn get_data(&self) -> Self::Data;
6     fn get_bytes(&self) -> [u8; N_BYTES];
7 }
```

Cecha `LabeledReadout` została zaimplementowana dla struktury `TypedLabelReadout`. Dodatkowo utworzono strukturę `DynamicLabeledReadout` udostępniającą metodę `from_bytes` konstruującą obiekt z bajtów. Struktura ta nie została zastosowana w samym projekcie, ale okazała się przydatna podczas konstrukcji odbiornika.

5.8 Czujnik klimatyczny

Centralnym elementem warstwy odpowiedzialnej za pomiary środowiskowe jest struktura `ClimateSensor`, której zadaniem jest agregacja trzech niezależnych modułów pomiarowych — czujnika temperatury i wilgotności AHT20, czujnika ciśnienia i temperatury BMP280 oraz czujnika natężenia oświetlenia VEML7700 — pod wspólnym, jednolitym interfejsem. Struktura ta hermetyzuje nie tylko same sterowniki poszczególnych czujników, lecz również współdzieloną magistralę komunikacyjną I²C, mechanizm opóźnień (`DelayNs`) oraz piny przetwornika ADC wykorzystywane do pomiaru napięcia

na kondensatorach układu zasilania. Dzięki temu logika odpowiedzialna za obsługę pojedynczego węzła pomiarowego została skoncentrowana w jednym miejscu, co znacząco upraszcza jej dalsze wykorzystanie w warstwie wyższego poziomu, odpowiedzialnej za transmisję danych.

Parametryzacja generyczna struktury (`VemlConfig`, `Bmp280Config`, `D`, `SumCapPin`, `SecondCapPin`) pozwala na statyczne, rozstrzygane na etapie kompilacji dostosowanie konfiguracji poszczególnych czujników oraz przypisanych pinów, bez wprowadzania narzutu związanego z dynamicznym dyspozytorem (ang. *dynamic dispatch*).

Za centralizację odczytu odpowiadają metody `read_modules` oraz `read_charge_info`, które sekwencyjnie odpytują poszczególne czujniki, a uzyskane wartości koduje przy użyciu typu `TypedLabelReadout`, zapisując je bezpośrednio do wspólnego bufora bajtowego. Metoda `read_bytes`, stanowiąca publiczny punkt wejścia dla warstw nadrzędnych, łączy identyfikator węzła pomiarowego, dane z modułów klimatycznych oraz informacje o stanie naładowania kondensatorów w jedną, spójną ramkę danych o ustalonym rozmiarze, gotową do dalszej transmisji.

Obsługa błędów została zorganizowana w sposób hierarchiczny, poprzez zestaw dedykowanych typów wyliczeniowych (`ModuleInitError`, `ClimateSensorInitError`, `ModuleReadError`, `ClimateSensorReadError`) wraz z implementacjami cechy `From`, umożliwiającymi automatyczną propagację błędów niższego poziomu za pomocą operatora `?`.

5.9 Usypianie mikrokontrolera

Moduł `sleep` odpowiada za zarządzanie energooszczędnym trybem pracy mikrokontrolera, wykorzystując do tego celu Watchdog Timer (WDT) skonfigurowany w trybie przerwania, a nie resetu. Funkcja `setup_wdt` inicjalizuje licznik z interwałem wynoszącym ok. 8 sekund oraz włącza bit `WDIE`, dzięki czemu każde przepełnienie licznika generuje przerwanie, a nie sprzętowy restart układu. Ponieważ pojedynczy cykl WDT jest zbyt krótki dla docelowego okresu uśpienia, licznik `WDT_COUNT` — chroniony przez `interrupt::Mutex` ze względu na dostęp z kontekstu przerwania — zlicza kolejne wybudzenia, aż osiągnie wartość `WAKE_CYCLES`, co odpowiada łącznemu czasowi uśpienia równemu ok. minucie. Funkcja `ready` udostępnia ten stan warstwie wyższego poziomu, zwracając `true` i zerując licznik dopiero po osiągnięciu zadanej liczby cykli.

Właściwe przejście w stan uśpienia realizowane jest przez funkcję `enter_sleep`, która przed uśpieniem wyłącza zbędne peryferia (timery, USART, SPI, TWI) za pomocą rejestru Power Reduction Register, pozostawiając aktywny jedynie przetwornik ADC. Mikrokontroler ustawiany jest w tryb power-down (najgłębszy dostępny tryb oszczędzania energii), a dodatkowo wyłączany jest detektor spadku napięcia za-

silania (BOD), co pozwala ograniczyć pobór prądu w czasie snu. Operacja ta wymaga zachowania ściśle określonej sekwencji zapisów do rejestru MCUCR w krótkim oknie czasowym, dlatego jest wykonywana z wyłączonymi przerwaniem globalnymi (`interrupt::free`).

Bezpośrednio po wywołaniu instrukcji uśpienia (`avr_device::asm::sleep()`) procesor zatrzymuje wykonywanie do momentu wystąpienia przerwania — w tym przypadku pochodzącego od WDT. Obsługa przerwania WDT inkrementuje licznik `WDT_COUNT` oraz ponownie ustawia bit `WDIE`, który jest automatycznie czyszczony przez sprzęt po każdym wyzwoleniu, co jest niezbędne do zachowania trybu przerwania (a nie resetu) przy kolejnych cyklach. Po powrocie ze stanu uśpienia funkcja `enter_sleep` czyści bit `SE` (Sleep Enable) oraz przywraca zasilanie wszystkich peryferiów, kończąc cykl.

5.10 Sterowanie zasilaniem

Struktura `PowerManager` odpowiada za sterowanie zasilaniem peryferiów o podwyższonym poborze prądu (moduł RF, licznik Geigera, magistrala I2C), poprzez niezależne przełączanie odpowiadających im pinów cyfrowych skonfigurowanych jako wyjścia. Typy pinów są parametrami generycznymi, co pozwala uniknąć narzutu dynamicznego dyspozytora przy jednoczesnym zachowaniu elastyczności konfiguracji sprzętowej.

Istotnym elementem implementacji jest obsługa flagi `active_low`, uwzględniającej fakt, że w zależności od zastosowanego układu sterującego zasilaniem (np. tranzystora), stan aktywny modułu może odpowiadać zarówno stanowi wysokiemu, jak i niskiemu na pinie sterującym. Dzięki zastosowaniu operacji XOR (^) pomiędzy pożądanym stanem logicznym a wartością tej flagi, interfejs udostępniany przez metody `activate_*/deactivate_*` pozostaje niezależny od szczegółów elektrycznych konkretnego rozwiązania sprzętowego.

Funkcjonalność została podzielona na dwa poziomy granularności: metody `activate_power_hungry/deactivate_power_hungry` sterują wyłącznie modułem RF, którego pobór prądu jest na tyle duży, że wymaga on osobnej, częstszej kontroli niezależnie od pozostałych czujników, natomiast metody `activate_all/deactivate_all` zbiorczo przełączają wszystkie zarządzane piny za pośrednictwem prywatnej metody pomocniczej `set_all_state`, co pozwala na jednoczesne, całkowite odcięcie zasilania peryferiów.

5.11 Funkcja główna

Funkcja `main` stanowi punkt centralny aplikacji, spinający w jedną całość wszystkie opisane wcześniej moduły w ramach głównej pętli programu, zorientowanej na ograniczenie zużycia energii. Po jednorazowej inicjalizacji peryferiów (ADC, magistrali I2C, nadajnika radiowego OOK oraz obiektu `PowerManager`), a także uzbrojeniu liczników sprzętowych (WDT oraz timera nadajnika radiowego), urządzenie przechodzi we właściwy cykl pracy, w którym większość czasu spędza w stanie uśpienia — wybudzane cyklicznie jedynie przez przerwanie WDT, aż `sleep::ready()` zasygnalizuje upływ zadanego okresu (1 minuty).

Po każdym wybudzeniu następuje krótkotrwała aktywacja modułów o podwyższonym poborze prądu (za pośrednictwem metody `power_manager.activate_power_hungry()`) oraz, przy braku wcześniejszej inicjalizacji, jednorazowa inicjalizacja czujnika klimatycznego. Odczytane dane pomiarowe są następnie wielokrotnie (`SEND_DATA_N_TIMES`) nadawane przez transponder radiowy, przy czym błędy zarówno inicjalizacji, jak i odczytu czujnika są świadomie ignorowane (`continue`/pusty blok `Err`) — co jest podejściem uzasadnionym w kontekście urządzenia bateryjnego, pracującego bez nadzoru, gdzie priorytetem jest kontynuacja działania kosztem utraty pojedynczego pomiaru. Po zakończeniu transmisji zasilanie modułów energochłonnych jest ponownie wyłączane, a stan diody statusowej sygnalizuje aktywność urządzenia w trakcie całego cyklu pomiarowego.

Osobno zdefiniowana została procedura obsługi paniki (`panic_handler`), która — z uwagi na brak możliwości bezpiecznego kontynuowania działania programu — sygnalizuje błąd za pomocą migającej diody LED, a następnie wymusza twardy restart mikrokontrolera poprzez skonfigurowanie WDT w trybie resetu sprzętowego z minimalnym możliwym czasem oczekiwania.

Rozdział 6

Testy

6.1 Zasięg komunikacji radiowej

6.2 Pobór prądu

6.3 Czas pracy

6.4 Porównanie z rozwiązaniami komercyjnymi

Podsumowanie

Tu treść podsumowania (którą — oczywiście — też piszemy na końcu).

Spis listingów

5.1	Inicjalizacja modułu AHT20	29
5.2	Odczyt surowych danych z modułu AHT20	29
5.3	Konwersja surowych danych z modułu AHT20	31
5.4	Sprawdzanie sumy kontrolnej danych modułu AHT20	31
5.5	Odczyt danych modułu AHT20	32
5.6	Struktura <code>Bmp280</code>	33
5.7	Odczyt danych kompensacyjnych dla modułu BMP280	33
5.8	Cecha <code>Config</code> używana do konfiguracji BMP280	35
5.9	Przykładowa implementacja cech konfiguracyjnych modułu BMP280 . .	35
5.10	Inicjalizacja modułu BMP280	36
5.11	Odczyt surowych danych z modułu BMP280	37
5.12	Funkcja <code>read</code> struktury <code>Bmp280</code>	38
5.13	Struktura <code>Veml7700</code>	39
5.14	Cecha <code>Config</code> dla VEML7700	40
5.15	Przykładowa konfiguracja dla modułu VEML7700	41
5.16	Funkcja <code>init</code> struktury <code>Veml7700</code>	41
5.17	Konwersja surowych danych modułu VEML7700 na wartość wyrażoną w luksach	42
5.18	Odczyt danych z modułu VEML7700	43
5.19	Struktura <code>GeigerCounter</code>	43
5.20	Metoda <code>init</code> struktury <code>GeigerCounter</code>	44
5.21	Metoda <code>init</code> struktury <code>GeigerCounter</code>	44
5.22	Metoda <code>cpm</code> struktury <code>GeigerCounter</code>	45
5.23	Struktura <code>Transmitter</code>	47
5.24	Metoda <code>send</code> struktury <code>Transmitter</code>	48
5.25	Struktura <code>Transmitter</code>	50
5.26	Implementacja struktury <code>TypedLabelReadout</code>	50
5.27	Struktura <code>Transmitter</code>	51

Spis tabel

1.1	Zestawienie wybranych modułów pomiarowych dostępnych na rynku . .	10
2.1	Porównanie języków programowania w systemach wbudowanych	16
4.1	Napięcie wyjściowe U_{wy} regulatora bocznikowego przy zastosowaniu diody Zenera i diody LED, dla prądu emitera I_E tranzystora zwierającego. Do testu użyto zasilacza stałoprądowego	22
4.2	Funkcje wyprowadzeń mikrokontrolera	23

Spis rysunków

4.1	Sekcja ładowania superkondensatorów z użyciem diod zenera (D1, D2)	22
4.2	Sekcja ładowania superkondensatorów z użyciem diod LED (D1, D2)	22
4.3	Układ regulacji napięcia oparty o przetwornicę step-up i regulator liniowy	23
4.4	Mikrokontroler ATmega328P widoczny na schemacie projektu	24
4.5	Tranzystor załączający linię zasilania urządzeń magistrali I ² C	25
4.6	Konwertery poziomów logicznych dla linii SDA i SCL magistrali I ² C	25

Bibliografia

- [1] ALLPCB. Top microcontrollers every engineer should know in 2025, 2025.
- [2] Anonimowy użytkownik forum Elektroda.pl. Licznik geigera (bez znienawidzonego MC34063) o poborze prądu $< 3\text{mA}$. <https://www.elektroda.pl/rtvforum/topic3582237.html>, 2026. Forum internetowe Elektroda.pl; dostęp: 26 maja 2026.
- [3] Aosong Electronics. Dht11 technical data sheet, 2018.
- [4] Aosong Electronics. Dht22 (am2302) digital temperature and humidity sensor datasheet, 2018.
- [5] Asair - Guangzhou Aosong Electronics Co., Ltd. *AHT20 Product manuals*, 2019.
- [6] Atmel Corporation. *ATmega328P*, 2015.
- [7] AUTOSAR. Guidelines for the use of the c++14 language in critical and safety-related systems. Adaptive Platform Specification Document ID 839, AUTOSAR, October 2017.
- [8] Michael Barr and Anthony Massa. *Programming Embedded Systems: With C and GNU Development Tools*. O'Reilly Media, 2nd edition, 2006.
- [9] Barr Group. Embedded systems safety & security survey 2017, 2017.
- [10] Bosch Sensortec. Bmp180 digital pressure sensor datasheet, 2013.
- [11] Bosch Sensortec. Bmp388 digital pressure sensor datasheet, 2020.
- [12] Bosch Sensortec. Bmp390 digital pressure sensor datasheet, 2022.
- [13] Bosch Sensortec. Bme280 combined humidity and pressure sensor datasheet, 2024.
- [14] Bosch Sensortec. Bme680 environmental sensor, 2026.
- [15] Bosch Sensortec GmbH. *BMP280 Digital Pressure Sensor – Data Sheet*. Bosch Sensortec GmbH, revision 1.26 edition, October 2021. Dostęp: 2026-05-31.

- [16] Bresser. Weather stations & clocks, 2026.
- [17] Paul Clifford. I2C Bus Range and Electrical Specifications, Freescale 9S12 HCS12 MC9S12 I2C Hardware. <http://www.mosaic-industries.com/embedded-systems/sbc-single-board-computers/freescale-hcs12-9s12-c-language/instrument-control/i2c-bus-specifications>, 2016. Mosaic Industries. Accessed: 2026-06-02.
- [18] cppreference.com contributors. C++ reference, 2026.
- [19] Holtek Semiconductor Inc. *HT73XX Low Power Consumption LDO*.
- [20] Ken Research. Global microcontroller (mcu) market. Technical report, Ken Research, 2025. Market research report, accessed 2026-06-15.
- [21] Marcy Lowe, Saori Tokuoka, Tali Trigg, and Gary Gereffi. Lithium-ion batteries for electric vehicles: the u.s. value chain. 10 2010.
- [22] MicroPython Developers. Micropython: A lean and efficient python implementation for microcontrollers and constrained systems, 2026.
- [23] Alain Nakad, Mervat Madi, Olav Aaker, Efstratios Ntantis, and Karim Kabalan. Comparing supercapacitors to lithium-ion batteries through measurements and simulations. volume 2023, 11 2023.
- [24] Nanjing Micro One Electronics Inc. *DC/DC Step up Converter ME2108 Series*.
- [25] Netatmo. Smart weather station, 2026.
- [26] Oregon Scientific. Weather stations, 2026.
- [27] ROHM Semiconductor. Bh1750fvi digital ambient light sensor datasheet, 2011.
- [28] Robert C. Seacord. *Secure Coding in C and C++*. SEI Series in Software Engineering. Addison-Wesley Professional, 2 edition, 2013. PDF copy accessed via GitHub repository.
- [29] NXP Semiconductors. *UM10204 I2C-bus specification and user manual*. NXP Semiconductors, 10 2021.
- [30] Sensirion AG. Datasheet sht3x-dis humidity and temperature sensor, 2022.
- [31] Sensirion AG. Datasheet sht4x humidity and temperature sensor, 2025.

- [32] Klabnik Steve, Nichols Carol, and Krycho Chris. *The Rust Programming Language*. No Starch Press, 2023. Dostępne online: <https://doc.rust-lang.org/book/>, [Dostęp: 15.06.2026].
- [33] TFA Dostmann. Wireless weather stations, 2026.
- [34] TFA Dostmann. Wireless weather stations with 433 mhz and 868 mhz radio transmission, 2026.
- [35] Salauiyou Valery, Bułatowa Irena, Grześ Tomasz, and Bułatow Witali. *Podstawy projektowania systemów wbudowanych*. Oficyna Wydawnicza Politechniki Białostockiej, Białystok, 2024.
- [36] Vishay Semiconductors. VEML7700: High Accuracy Ambient Light Sensor With I²C Interface. Datasheet 84286, Vishay Semiconductors, 2017. Rev. 1.0.
- [37] Zephyr Project Contributors. 1-Wire Bus — Zephyr Project Documentation. <https://docs.zephyrproject.org/latest/hardware/peripherals/w1.html>, 2026. Last source update: Feb 09, 2026. Accessed: 2026-06-02.